Relatório de MQTT e Dashboard

Projeto CardioIA - Grupo PulseIA
Fase 3: Monitoramento Contínuo - IoT no Pulso do Paciente

Visão geral da comunicação

A segunda parte do protótipo leva as leituras processadas no ESP32 até uma camada de visualização. O dispositivo cria uma mensagem JSON para cada amostra e a publica no tópico MQTT `cardioia/pulseia/v1/telemetria`. O broker funciona como intermediário: recebe a publicação do ESP32 e entrega a mensagem aos clientes inscritos no mesmo tópico. O Node-RED é esse cliente e representa a camada Fog da atividade, pois transforma a telemetria em componentes de visualização próximos da aplicação.

O fluxo adotado é: sensores simulados → fila no ESP32 → Wi-Fi → broker MQTT → Node-RED → dashboard. A fila do dispositivo é importante porque o envio pode falhar; nenhuma mensagem é removida antes do retorno positivo da função de publicação. O broker público configurado no exemplo permite demonstrar o fluxo sem expor uma conta. Para qualquer ambiente além da demonstração, a configuração deve ser substituída por um broker privado, credenciais individuais e TLS.

Payload e tópico

O payload contém um identificador demonstrativo do paciente, momento relativo da leitura, temperatura, umidade, BPM, indicação de pulso, estado de alerta e tamanho da fila offline, contador de descartes e sequência da amostra. O uso de JSON facilita a leitura no Node-RED e mantém as variáveis nomeadas. O identificador `PACIENTE_DEMO_001` é fictício; não há dados de pacientes reais neste projeto.

Campo	Uso no protótipo
`temperatura_c`	Medidor de temperatura e regra de alerta
`umidade_pct`	Medidor complementar para contexto ambiental
`bpm`	Gráfico temporal e regra de alerta
`alerta`	Sinalização no painel
`fila_offline`	Evidência da resiliência no Edge
`sequencia`	Ordem das amostras e apoio à identificação de lacunas
`amostras_descartadas`	Transparência sobre perdas por limite local
`origem`	Identificação da fonte da telemetria

Configuração MQTT

O código usa a biblioteca PubSubClient. Durante a demonstração, `config.h` aponta para `broker.hivemq.com` na porta 1883, sem autenticação. Essa escolha é restrita ao exercício e ao tópico de telemetria fictícia. O nome do tópico foi criado especificamente para o CardioIA e não deve transportar qualquer dado pessoal ou informação de saúde identificável.

O arquivo `config.h.example` contém os parâmetros para uma conta no HiveMQ Cloud: host do cluster, porta 8883, usuário, senha e certificado raiz. Com `MQTT_USAR_TLS` definido como `1`, o programa só permite a conexão se houver certificado raiz configurado. O HiveMQ documenta a porta 8883 com TLS para clusters

Cloud e recomenda TLS para proteger o canal de comunicação: configuração de broker e segurança.

Nenhuma credencial foi incluída no repositório. A configuração pessoal fica em `config.local.h`, ignorado pelo Git. O buffer MQTT foi ampliado para 512 bytes, evitando que o payload e o tópico ultrapassem o limite padrão da biblioteca.

Dashboard Node-RED

O arquivo `node-red/flows_cardioia.json` pode ser importado diretamente no Node-RED. Ele contém um nó MQTT In, conversão de JSON, uma função que distribui as propriedades e componentes do pacote `node-red-dashboard`. O fluxo tem um gráfico de BPM, um gauge de temperatura, um gauge de umidade, um texto de alerta e um texto que informa quantas amostras aguardam na fila local.

Antes de distribuir os valores, a função do Node-RED verifica se o payload é um objeto e se BPM, temperatura e umidade são números válidos. Mensagens inválidas são descartadas com aviso no editor. Dois nós de teste permitem enviar uma leitura normal e outra em alerta sem depender do ESP32, o que facilita validar a interface e demonstrar suas regras. A sequência mais recente é mantida no contexto do fluxo; se a próxima mensagem saltar números, o indicador do Edge informa quantas amostras estão ausentes.

O alerta aparece como `ALERTA SIMULADO: <BPM> BPM | <temperatura> °C` quando BPM é maior ou igual a 120 ou temperatura é maior ou igual a 38 °C. Caso contrário, exibe `Sem alerta pelos limites`. A mensagem foi escrita como orientação de verificação e não como diagnóstico: leitura isolada, sensor simulado e limite fixo não permitem inferir condição médica.

Para executar, instala-se o pacote `node-red-dashboard`, importa-se o arquivo, confere-se host, porta e tópico, faz-se Deploy e abre-se `/ui`. O arquivo node-red/README.md traz o procedimento e as evidências recomendadas. Caso seja utilizado HiveMQ Cloud, o nó de broker do Node-RED deve ser atualizado para TLS e credenciais antes do Deploy, sem salvar senhas no export compartilhado.

O `node-red-dashboard` clássico foi mantido nesta entrega por compatibilidade com o fluxo acadêmico e pela simplicidade de importação, mas o próprio projeto o identifica como descontinuado. Em uma evolução de produção, a interface deve ser migrada para o FlowFuse Dashboard (`@flowfuse/node-red-dashboard`). Essa decisão é registrada para não apresentar uma dependência legada como solução definitiva. Referência: aviso do mantenedor.

Evidências e testes demonstráveis

O teste normal consiste em iniciar o Wokwi com conectividade ativa e observar no Serial um JSON publicado, ao mesmo tempo em que o dashboard recebe valores. Para testar BPM, pressiona-se o botão `PULSO` repetidamente dentro da janela de dez segundos. Para demonstrar alerta de temperatura, altera-se a propriedade do DHT22 no Wokwi para um valor igual ou superior a 38 °C. O dashboard deverá atualizar o gauge e o texto de alerta.

O teste de resiliência começa com o botão `ONLINE/OFFLINE`: desative a rede, aguarde ao menos duas coletas e confira o contador da fila no Serial. Ao reativar a conectividade, o Serial deve mostrar cada mensagem sendo publicada e a fila deve chegar a zero. O valor publicado em `fila_offline` já representa o número estimado de pendências depois do envio atual, de modo que a última mensagem apresenta zero. Capturas dessas três situações comprovam, de forma visual, sensores, alertas, MQTT e sincronização offline.

Considerações finais

A arquitetura atende ao fluxo de IoT proposto porque separa captura, tratamento local, entrega por protocolo leve e visualização. Porém, MQTT não resolve por si só privacidade, segurança ou confiabilidade clínica. Em produção, o projeto precisaria de autenticação por dispositivo, TLS, autorização por tópicos, rotação de credenciais, logs auditáveis, monitoramento, retenção limitada de dados e validação de todas as regras de alerta por profissionais habilitados.